

The Energy-Accuracy Frontier: Benchmarking SNNs on Low-Resource Indian Linguistic Tasks

Priyam Saha¹, Debiprasanna Sahoo¹, Abhik Jana¹

¹IIT Bhubaneswar, India

{21cs01076, dpsahoo, abhikjana}@iitbbs.ac.in

Abstract

The increasing complexity of state-of-the-art neural networks creates significant computational energy overhead in Natural Language Processing (NLP) applications. A few studies have explored opportunities to reduce overhead by using a Spiking Neural Network (SNN) instead of a mainstream artificial neural network (ANN) in computer vision and sensory domains. A comprehensive analysis of SNN’s usability for NLP tasks in low-resource languages has yet to be conducted. Hence, in this paper, we present the first-ever comparative energy-overhead analysis of ANNs vs. SNNs for text classification tasks in low-resource Indian languages such as Hindi, Bengali, and Telugu. Our experiments show that although we lose classification accuracy by an acceptable margin when replacing an ANN with an SNN, we save significant energy, which highlights the potential for efficient utilization of edge computing devices to run NLP tasks for low-resource languages. This work shows a direction for advancing energy-efficient NLP applications, especially for low-resource languages. We make code and dataset publicly available ¹.

1 Introduction

With the emergence of Transformers (Vaswani et al., 2017), the domain of Natural Language Processing has been revolutionized. This has led to a substantial requirement for computational resources and hence, energy for training and deployment. The rise in model complexity poses challenges for the practical use of NLP tasks in resource-constrained environments, such as mobile devices and edge sensors (De Vries, 2023). Hence, efforts are underway to incorporate energy efficiency into Artificial Intelligence (AI) models.

Such efforts are primarily focused on quantization, pruning (Liang et al., 2021), and model distillation (Poddar et al., 2025). These techniques reduce computational overhead by a significant margin, while preserving dense, real-valued operations, as seen in traditional artificial neural networks (ANNs) (Lv et al., 2024).

In contrast, Spiking Neural Networks (SNNs) (Maass, 1997) present a fundamentally different, event-driven paradigm inspired by biological neurons, where computation occurs only in response to discrete spikes. This leads to considerable reductions in computational overhead and, hence, energy demands (Lv et al., 2024; Knipper et al., 2024). Although SNNs have demonstrated strong results with competitive accuracy and significant energy savings in vision and sensory domains (Yamazaki et al., 2022), the applicability of SNNs to NLP tasks remains relatively underexplored. In one recent work, Lv et al. (2024) attempt to compare computational efficiency through both floating-point operations (FLOPs) for ANNs and synaptic operations (SOPs) for SNNs, providing a clear, dual-metric framework. This work demonstrates that SNN-based text classifiers can also achieve accuracy close to that of their ANN (i.e., Convolution Neural Network) counterparts while exhibiting enhanced robustness to adversarial attacks. In another direction, Knipper et al. (2024) investigate methods for encoding text into spikes and found that deterministic rate-coding techniques boost SNN text classification accuracy significantly compared to stochastic Poisson rate-coding, narrowing the performance gap between SNNs and ANNs. Together, these two studies establish the techniques for applying SNNs to efficient and robust natural language processing settings.

In this paper, we extend these state-of-the-art approaches and investigate the accuracy-energy trade-offs in a Spiking Neural Network for text classifica-

¹<https://github.com/PS1076/Energy-Accuracy-Frontier>

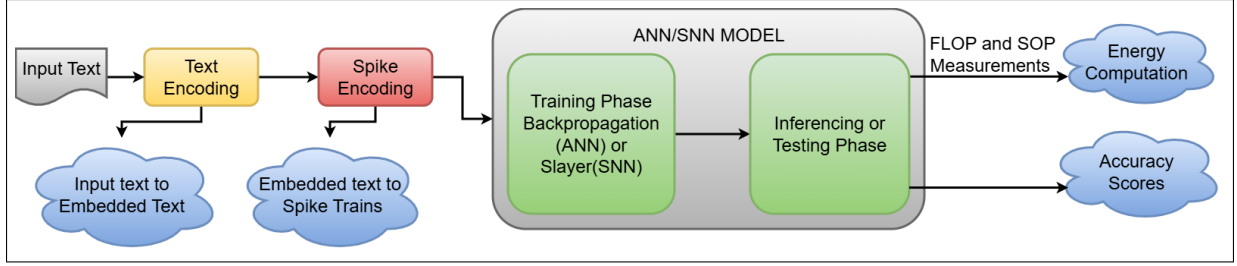


Figure 1: The flowchart of the pipeline used for our analysis.

tion tasks while dealing with low-resource Indian Languages like Hindi, Bengali, and Telugu. To ensure a fair comparison between ANN and SNN, we deploy a unified pipeline and measure the compute load using FLOPs (Floating-Point Operations) and SOPs (Synaptic Operations), respectively.

Our experiments show that, for the target low-resource languages, SNN reduces energy consumption by an average of 41.91x and 44.3x maximum compared to the ANN counterpart, while losing a minimal 0.895% in classification accuracy on average and a max of 3.99%. To the best of our knowledge, this is the first attempt to fairly compare the accuracy-energy trade-off of ANN and SNN for low-resource Indian languages. These findings will act as the stepping stone towards sustainable NLP, specifically in resource-constrained settings. We will make the code and data publicly available upon acceptance.

2 Proposed Pipeline and Workflow

The pipeline, as shown in Figure 1, consists of five key stages: Text embedding, Spike Train Encoding, Neural Model design (SNN/ANN), Training via backpropagation (for ANN) or surrogate gradients (for SNN), and Inference evaluation.

Text Embedding Input text (e.g., sentences, documents) is first converted into continuous-valued embeddings. For word-level tasks, pre-trained embeddings such as FastText (Bojanowski et al., 2017) are typically used, yielding 300-dimensional vectors. For sentence-level tasks, Transformer-based encoders such as Universal Sentence Encoder (USE) are employed to generate 512-dimensional vectors per sentence (Cer et al., 2018).

Spike Train Encoding A spike train is a temporal sequence of "spikes" (represented as 1s) and "no-spike" periods (represented as 0s). Unlike tra-

ditional artificial neural networks (ANNs) that communicate via continuous values, SNNs use these binary, event-driven signals to process data (Tavanaei et al., 2019).

Binarization (SNN-bin): Binarization reduces embeddings to binary (0/1) vectors using autoencoders or hash-based clustering. Here, each vector directly maps to either a spike or a non-spike event.

Poisson Rate Coding (SNN-rate-rand): In this method, we convert Floating-point values stochastically to spikes, simulating typical neural firing rates (Kalra and Sadanand, 2025).

Deterministic Rate Coding (SNN-rate): A self-accumulate-and-fire (SAF) neuron deterministically integrates an embedding component over time and generates spikes when a membrane threshold is crossed. This method provides a higher-fidelity representation of the input compared to stochastic Poisson coding, yielding notable accuracy improvements (Knipper et al., 2024). **Temporal Coding (e.g., Time-to-First-Spike):** The information is encoded in the precise timing of the first spike. Earlier spikes often represent stronger stimuli (Roy et al., 2019). In temporal coding, the network can often decide as soon as the first few spikes propagate through the layers, without waiting for the entire simulation window.

Neural Model Architecture We enforce strict architectural symmetry between the Artificial Neural Networks (ANNs) and Spiking Neural Networks (SNNs). Following the foundational framework proposed by Knipper et al. (Knipper et al., 2024), both paradigms are implemented as fully-connected, feedforward multilayer perceptrons. The dimensionality of the input layer is dynamically governed by the selected embedding strategy to accommodate various scales of linguistic representation: 512 neurons for continuous sentence-level embeddings, 300 for continuous word-level vectors, and 64 for binarized encoding

Language	Name	Class	Task	Source
Hindi	Sentiment Data(SDH)	7	Fine-grained emotion classification	SD source ¹
	IITP Product Review (PR)	3	Product review sentiment classification	Akhtar et al.(Akhtar et al., 2016)
	IITP Movie Review (MR)	3	Movie review sentiment classification	Akhtar et al.(Akhtar et al., 2016)
	BBC News Articles (BNA)	14	BBC Hindi news topic classification	BNA source ²
Bengali	Social Media Data (SMD)	2	Social media text classification	Hasan et al.(Hasan et al., 2023)
	Sentiment Data (SDB)	3	Standard sentiment analysis	Islam et al.(Islam et al., 2020)
	SentNoB (SNB)	3	Bengali text sentiment analysis	Islam et al.(Islam et al., 2021)
Telugu	Telugu sentiment analysis (TSA)	3	Sentiment analysis for Telugu corpus	Mukku et al.(Mukku and Mamidi, 2017)
	IndicNLP News (INN)	3	Telugu news classification	INN source ³
	INLTK Headlines (IH)	3	Telugu news headline classification	IN source ^{4,5}

Table 1: Data Set Description

schemes. These inputs propagate through a hierarchical feature-extraction pipeline with two hidden layers, empirically fixed at 256 and 128 neurons, respectively. The output layer size is strictly tailored to map to the number of target classes K dictated by the specific downstream classification task.

Crucially, while the macroscopic topologies of the networks remain isomorphic, their underlying computational mechanisms diverge significantly. The baseline ANN utilizes standard continuous-valued, floating-point dense layers paired with Rectified Linear Unit (ReLU) (Nair and Hinton, 2010) activation functions. In contrast, the SNN operates fundamentally in the temporal domain. All continuous activations are replaced with event-driven, discrete-spiking nodes. Specifically, the SNN hidden layers employ Current-Based Leaky Integrate-and-Fire (CUBA LIF) (Shrestha and Orchard, 2018) neuron dynamics, characterized by distinct voltage and current decay parameters or the proposed Spiking Activation Function (SAF) (Ahmed and Orrje, 2023) neurons.

Training and Optimization For ANN Training, conventional backpropagation is applied using cross-entropy loss whereas for SNN Training, surrogate gradient methods (Neftci et al., 2019) such as SLAYER (Shrestha and Orchard, 2018) are used. SLAYER is a training algorithm designed for Spiking Neural Networks (SNNs) that addresses the challenge of non-differentiable spike activations by using surrogate gradients. It treats spikes probabilistically and approximates the gradient of the spiking function to enable backpropagation through time (BPTT). (Lv et al., 2024). Both model types are trained for the same number of epochs (typically 50) with analogous hyperparameters. We use Adam optimizer with a learning rate of 1×10^{-5} along with 15 samples/batch and 50 epochs of training. For

backpropagation, a spike-rate loss function with SLAYER’s surrogate gradient method is deployed.

Inference A major component of the inference time is the Inference Window. It is a hyperparameter that must be set to allow sufficient time for spike integration (e.g., a 50ms window). This must be tuned appropriately to achieve competitive accuracy while significantly reducing the latency penalty. The output neuron with the highest spike rate over the inference window determines the final classification.

3 Details of Core Components

Leaky Integrate-and-Fire (LIF) Neuron Dynamics. The core computational element in the SNN model is the Leaky Integrate-and-Fire (LIF) spiking neuron, modeled with discrete-time dynamics (Lv et al., 2024)

$$U_t = I_t + \beta \cdot U_{t-1} - S_{t-1} \cdot U_{thr} \quad (1)$$

$$I_t = W \cdot X_t \quad (2)$$

$$S_t = \begin{cases} 1, & \text{if } U_t \geq U_{thr} \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

Where U_t is the membrane potential at timestep t ; I_t is the input current, calculated by the weighted sum of inputs X_t with synaptic weights W ; β is the voltage decay factor (corresponding to membrane leak); S_t is the spike output, a binary signal indicating neuron firing when membrane potential crosses the threshold U_{thr} .

¹<https://www.kaggle.com/datasets/praths71018/hindi-sentiment-dataset/data>

²<https://github.com/NirantK/hindi2vec/releases/tag/bbc-hindi-v0.1>

³https://github.com/AI4Bharat/indicnlp_catalog

⁴<https://github.com/goru001/inltk>

⁵https://github.com/AI4Bharat/indicnlp_catalog

Language	Hindi (Hi)				Bengali (Bn)			Telugu (TI)		
Dataset	SDH	PR	MR	BNA	SMD	SDB	SNB	TSA	INN	IH
ANN Accuracy (%) (↑)	46.33	56.00	42.22	39.69	88.93	62.32	61.95	47.03	88.04	46.13
SNN Accuracy (%) (↑)	46.83	54.29	41.91	47.42	85.47	58.33	58.99	44.32	86.00	46.13
ANN FLOPs ($\times 10^8$)	1.33	0.57	0.34	0.72	2.56	1.51	1.74	0.59	2.64	0.59
ANN Energy ($\downarrow$) ($\times 10^7$ pJ)	165.69	71.88	42.61	90.48	319.73	188.71	217.98	74.36	329.86	74.36
SNN SOPs ($\times 10^8$)	4.96	2.25	1.32	2.65	10.90	5.89	6.83	2.35	9.76	2.31
SNN Energy ($\downarrow$) ($\times 10^7$ pJ)	3.82	1.74	1.02	2.04	8.36	4.53	5.26	1.81	7.51	1.78
Accuracy Drop (%) ANN $\rightarrow$ SNN	-0.5	1.71	0.31	-7.73	3.46	3.99	2.96	2.71	2.04	0
Energy Eff. Ratio ($\times$)	43.40	41.41	41.95	44.30	38.26	41.63	41.42	41.06	43.90	41.78

Table 2: Performance of ANN and SNN in terms of Accuracy and Energy Comparisons for text classification task across target languages. Hi $\rightarrow$ Hindi; Bn $\rightarrow$ Bengali; TI $\rightarrow$ Telugu.

Language	Hindi (Hi)				Bengali (Bn)			Telugu (TI)			
Dataset	SDH	PR	MR	BNA	SMD	SDB	SNB	TSA	INN	IH	Mean
$\bar{\gamma}$	0.373	0.395	0.388	0.368	0.426	0.390	0.393	0.398	0.370	0.392	≈ 0.389

Table 3: Firing rate ($\bar{\gamma}$) per dataset

Upon spiking, the membrane potential is reset by subtracting the threshold U_{thr} (soft reset). The LIF neuron acts as a stateful unit accumulating inputs over discrete simulation timesteps and emitting spikes based on internal dynamics.

Rate Coding Method. Processing the continuous-valued input representations like word or sentence embeddings within the spiking domain requires employing a deterministic rate-coding technique known as the Self-Accumulate-and-Fire (SAF) method (Ahmed and Orrje, 2023). Unlike stochastic encoding schemes, the SAF neuron deterministically accumulates the normalized input value at each discrete time step and triggers a spike exactly when the neuron’s membrane threshold is exceeded. The temporal dynamics of this encoding mechanism are mathematically defined as follows:

$$V_t = V_{t-1} + x \quad (4)$$

$$Y_t = \begin{cases} 1, & \text{if } V_t \geq \alpha \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

To preserve the residual potential and maintain a high-fidelity representation of the input value, the membrane potential undergoes a subtractive reset immediately following a spike emission (when $Y_t = 1$):

$$V_t \leftarrow V_t - \alpha \quad (6)$$

In this formulation: x represents the static input value from the specific embedding dimension being encoded, strictly normalized to the range $[0, 1]$. V_t denotes the internal membrane potential

of the encoding neuron, which steadily integrates the input x over successive time steps. α is the predefined neuron firing threshold that dictates the exact condition for encoding a spike. Y_t is the generated output spike train, forming a temporal binary sequence where 1 indicates an emitted spike and 0 indicates a resting state.

Energy Estimation Process. *Floating Point Operations (FLOPs)* (Lv et al., 2024) refer to the total number of arithmetic operations involving floating-point numbers performed by a model, including multiplications and additions, typically used to measure computational complexity and cost. On the other hand, *Synaptic Operations (SOPs)* (Hu et al., 2021; Indiveri et al., 2015) are the count of operations in a spiking neural network where energy is consumed only when spikes occur; The number of SOPs in an SNN layer is estimated by:

$$\text{SOPs}_{\text{layer}} = T \times \text{firing_rate}_{\text{layer}} \times \text{FLOPs}_{\text{layer}} \quad (7)$$

where, T is the number of discrete simulation timesteps ($T = 10$ timesteps for all datasets); $\text{firing_rate}_{\text{layer}}$ is the average neuron firing rate in a layer, defined as the fraction of active spikes in the input tensor to that layer, measured directly during inference via forward hooks. For each dataset, we also compute overall *firing rate* $\bar{\gamma}$ (averaged across the 3 layers), which summarizes how active the spiking network’s neurons are on average. The details are presented in Table 3. $\text{FLOPs}_{\text{layer}}$ is the number of floating-point operations required for the equivalent ANN layer. The SNN energy

consumption per layer:

$$E_{\text{SNN,layer}} = \text{SOP}_{\text{SNN,layer}} \times E_{\text{SOP}} \quad (8)$$

where $E_{\text{SOP}} = 77$ femtojoules is the theoretical energy cost per synaptic operation, as reported in the literature (Lv et al., 2024). On the other hand, the ANN energy consumption per layer:

$$E_{\text{ANN,layer}} = \text{FLOP}_{\text{ANN,layer}} \times E_{\text{FLOP}} \quad (9)$$

where $E_{\text{FLOP}} = 12.5$ picojoules is the theoretical energy cost per floating-point operation (Lv et al., 2024).

4 Experimental Results and Analysis

For our experiments, as shown in Figure 1, we deploy the pipeline in word-level, rate-coded, and deterministic SNN mode for various classification tasks. This choice is grounded in recent studies (Tavanai et al., 2019; Kim et al., 2022) that highlight the significant energy advantages of spiking models and the robust performance of rate-coded inputs in text-based tasks. Specifically, we utilize the fully-connected (MLP) SNN configuration, with deterministic rate-coding of word embeddings.

Dataset Description. Table 1 shows 10 datasets used in our experiments.

Data Preprocessing and Embedding. We preprocess the input text segments to lowercase, remove the punctuations, and normalize them to preserve script-specific characters for potential multilingual use. Each input token is mapped to a pre-trained 300-dimensional FastText word embedding (Bojanowski et al., 2017), enabling the model to capture both semantic and subword information inherent in the text. For SNN input, we normalize these embeddings to the range $[0, 1]$ and convert them to spike trains via the SAF deterministic coding scheme. The spike trains accumulate the normalized value at each time step and trigger spikes when the neuron membrane threshold is exceeded.

Classification Accuracy. We report classification accuracies (for ANN and SNN) for all three target low-resource languages in Table 2. We observe that for Hindi, Bengali, and Telugu, the drop in accuracy is marginal (at most 3.99%). Note that for two Hindi datasets (SD and BNA), SNN even outperforms ANN. The SNN performs the same as the ANN on one of the Telugu datasets (IH).

Energy Profiling. We conduct our experiments using two NVIDIA 48 GB L40 GPUs. During

inference, we compute the number of Synaptic Operations (SOPs) for each layer, enabling us to calculate the total SOPs across all layers. Next, we utilize the aggregated SOP count to estimate the approximate computational energy consumption of the SNN. Similarly, for the ANN, we compute the layer-wise count of Floating Point Operations (FLOPs), then aggregate them across all layers to determine the total FLOP count. We use the total FLOP count to approximate the ANN model’s overall computational energy expenditure. Table 2 presents the computational energy results derived from these measurements. Energy consumption results show consistent savings of 38.26x to 44.30x using SNN compared to ANN baselines, due to sparse firing ($\text{firing_rate} \ll 1$) and lower energy per synaptic operation. This shows the direction towards sustainable NLP for low-resource Indian languages.

5 Conclusion and Future Work

In this paper, we investigated using SNNs (instead of ANNs) for sustainable, energy-efficient NLP in low-resource languages. We focused on text classification, with Hindi, Bengali, and Telugu as our target low-resource languages. We conducted a comprehensive study to examine the accuracy-energy trade-off for ANNs and SNNs. The experiments showed that SNN saves a significant amount of energy (up to 44.30x) while incurring no more than 4% accuracy loss across a relatively large number of target languages. Note that, our experiments use identical configurations to isolate architectural effects, but each model family may benefit from its own hyperparameters and training duration.

Hence, going forward we plan to tune ANNs and SNNs independently under an equal tuning budget, including SNN-specific parameters such as timesteps, decay factor, and firing threshold, and train each to convergence. We will also account for training cost alongside inference energy, compare models at matched accuracy and matched energy, and explore ANN-to-SNN conversion and hybrid training to further close the accuracy gap. We would also like to extend SNN methods to transformer-like architectures, hardware-aware energy modeling, including memory operations, and to investigate different genres of NLP tasks in the near future.

References

- Hisham Ahmed and Martin Orrje. 2023. Comparison of output decoding techniques for spiking neural networks.
- Md Shad Akhtar, Asif Ekbal, and Pushpak Bhat-tacharyya. 2016. Aspect based sentiment analysis in hindi: resource creation and evaluation. In *Proceedings of the tenth international conference on language resources and evaluation (LREC'16)*, pages 2703–2709.
- Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. Enriching word vectors with subword information. *Transactions of the association for computational linguistics*, 5:135–146.
- Daniel Cer, Yinfei Yang, Sheng-yi Kong, Nan Hua, Nicole Limtiaco, Rhomni St. John, Noah Constant, Mario Guajardo-Cespedes, Steve Yuan, Chris Tar, Brian Strope, and Ray Kurzweil. 2018. Universal sentence encoder for english. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 169–174.
- Alex De Vries. 2023. The growing energy footprint of artificial intelligence. *Joule*, 7(10):2191–2194.
- Md Arif Hasan, Firoj Alam, Anika Anjum, Shudipta Das, and Afiyat Anjum. 2023. Blp-2023 task 2: Sentiment analysis. *arXiv preprint arXiv:2310.16183*.
- Yangfan Hu, Huajin Tang, and Gang Pan. 2021. Spiking deep residual networks. *IEEE Transactions on Neural Networks and Learning Systems*, 34(8):5200–5205.
- Giacomo Indiveri, Federico Corradi, and Ning Qiao. 2015. Neuromorphic architectures for spiking deep neural networks. In *2015 IEEE International Electron Devices Meeting (IEDM)*, pages 4–2. IEEE.
- Khondoker Ittehadul Islam, Md Saiful Islam, and Md Ruhul Amin. 2020. Sentiment analysis in bengali via transfer learning using multi-lingual bert. In *2020 23rd International Conference on Computer and Information Technology (ICCIT)*, pages 1–5. IEEE.
- Khondoker Ittehadul Islam, Sudipta Kar, Md Saiful Islam, and Mohammad Ruhul Amin. 2021. Sentnob: A dataset for analysing sentiment on noisy bangla texts. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 3265–3271.
- Archit Kalra and Midhun Sadanand. 2025. Word2spike: Poisson rate coding for associative memories and neuromorphic algorithms. *arXiv preprint arXiv:2509.07361*.
- Youngeun Kim, Hyoungseob Park, Abhishek Moitra, Abhiroop Bhattacharjee, Yeshwanth Venkatesha, and Priyadarshini Panda. 2022. Rate coding or direct coding: Which one is better for accurate, robust, and energy-efficient spiking neural networks? In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 71–75. IEEE.
- R Alexander Knipper, Kaniz Mishty, Mehdi Sadi, and Shubhra Kanti Karmaker Santu. 2024. Snnlp: energy-efficient natural language processing using spiking neural networks. *arXiv preprint arXiv:2401.17911*.
- Tailin Liang, John Glossner, Lei Wang, Shaobo Shi, and Xiaotong Zhang. 2021. Pruning and quantization for deep neural network acceleration: A survey. *Neurocomputing*, 461:370–403.
- Changze Lv, Jianhan Xu, and Xiaoqing Zheng. 2024. [Spiking convolutional neural networks for text classification](#). *Preprint*, arXiv:2406.19230.
- Wolfgang Maass. 1997. Networks of spiking neurons: the third generation of neural network models. *Neural networks*, 10(9):1659–1671.
- Sandeep Sricharan Mukku and Radhika Mamidi. 2017. Actsa: Annotated corpus for telugu sentiment analysis. In *Proceedings of the first workshop on building linguistically generalizable NLP systems*, pages 54–58.
- Vinod Nair and Geoffrey E. Hinton. 2010. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on International Conference on Machine Learning, ICML'10*, page 807–814, Madison, WI, USA. Omnipress.
- Emre O Neftci, Hesham Mostafa, and Friedemann Zenke. 2019. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63.
- Soham Poddar, Paramita Koley, Janardan Misra, Niloy Ganguly, and Saptarshi Ghosh. 2025. Towards sustainable nlp: Insights from benchmarking inference energy in large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 12688–12704.
- Deboleena Roy, Priyadarshini Panda, and Kaushik Roy. 2019. [Synthesizing images from spatio-temporal representations using spike-based backpropagation](#). *Frontiers in Neuroscience*, Volume 13 - 2019.
- Sumit B Shrestha and Garrick Orchard. 2018. Slayer: Spike layer error reassignment in time. *Advances in neural information processing systems*, 31.
- Amirhossein Tavanaei, Masoud Ghodrati, Saeed Reza Kheradpisheh, Timothée Masquelier, and Anthony Maida. 2019. Deep learning in spiking neural networks. *Neural networks*, 111:47–63.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.

Kashu Yamazaki, Viet-Khoa Vo-Ho, Darshan Bulsara, and Ngan Le. 2022. Spiking neural networks and their applications: A review. *Brain sciences*, 12(7):863.